

Graph Data Structures & Algorithms

Complete FAANG / MAANG Interview Guide

Beginner to Advanced | 8 Phases | 30+ Problems

Tricks, Traces, Templates & Interview Tips

Personal Study Guide - 2026

TABLE OF CONTENTS

- 1. Roadmap Overview - 8 Phases at a Glance
- 2. Foundation - Graph Representations (Adjacency List & Matrix)
- 3. Phase 1 - BFS, DFS & Connectivity Problems
- 4. Phase 1.5 - Cycle Detection (Directed & Undirected)
- 5. Phase 2 - Topological Sort & Course Scheduling
- 6. Phase 2.5 - Clone Graph Pattern
- 7. Phase 3 - Shortest Path Algorithms (BFS, Dijkstra, Bellman-Ford, Floyd-Warshall)
- 8. Phase 4 - Minimum Spanning Tree (Prim's & Kruskal's)
- 9. Phase 5 - Advanced DFS (Bridges, Articulation Points, Tarjan's)
- 10. FAANG Problem Set - 30+ Must-Solve Problems by Phase
- 11. Master Cheat Sheet - All Tricks, Templates & Decision Trees

1. ROADMAP OVERVIEW - 8 Phases at a Glance

This roadmap takes you from zero graph knowledge to FAANG-ready in a structured progression. Each phase builds on the previous one.

Phase	Topic	Key Algorithms	Status
1	BFS / DFS / Connectivity	BFS, DFS, Flood Fill, Multi-src BFS	Done
1.5	Cycle Detection	Parent-track, Rec-stack, Kahn's, UF	Done
2	Topological Sort	Kahn's BFS, DFS Topo Sort	Done
2.5	Clone Graph	HashMap + DFS/BFS	Done
3	Shortest Path	Dijkstra, Bellman-Ford, Floyd-Warshall	Done
4	MST	Prim's, Kruskal's, Union-Find (DSU)	Done
5	Advanced DFS	Bridges, Articulation Pts, Tarjan's SCC	In Progress
6	Bipartite	2-coloring BFS/DFS	Pending
7	Adv. Shortest Path	A*, 0-1 BFS, Johnson's	Pending
8	Graph DP / SCC	SCC + DP, Euler Path	Pending

Learning Order (Follow This Exactly)

Build Graph -> BFS/DFS -> Connectivity -> Cycle Detection -> Topological Sort -> Clone Graph -> Shortest Path (BFS -> Dijkstra -> Bellman-Ford -> Floyd-Warshall) -> MST (Prim's -> Kruskal's) -> Bridges/Articulation -> Bipartite -> Advanced

2. FOUNDATION - Graph Representations

The 2x2 Decision Grid

Every graph has two independent properties. Pick one from each row:

Property	Option A	Option B
Direction	Directed (one-way)	Undirected (two-way)
Weight	Unweighted (0/1)	Weighted (has cost)

This gives 4 types: Directed-Unweighted, Directed-Weighted, Undirected-Unweighted, Undirected-Weighted. Each can be stored as Adjacency List or Adjacency Matrix = 8 total combinations.

MASTER TRICK: "Direction decides HOW MANY times you write. Weight decides WHAT you write."

Adjacency List vs Adjacency Matrix

Aspect	Adjacency List	Adjacency Matrix
Storage	$O(V + E)$	$O(V^2)$
Edge Lookup	$O(\text{degree})$	$O(1)$
Add Edge	$O(1)$	$O(1)$
Best For	Sparse graphs	Dense graphs, $V < 1000$
Interview Default	Almost always this	Only if asked

6 Memory Tricks for Graph Construction

- 1. Mirror Rule: Directed = 1 line of code (`graph[u].append(v)`). Undirected = mirror it (2 lines: `graph[u].append(v)`, `graph[v].append(u)`).
- 2. Count Your Appends: 1 append = directed. 2 appends = undirected.
- 3. Backpack Analogy: Each node has a backpack. Unweighted = note "I know node X". Weighted = note "I know X, distance=5".
- 4. Matrix Symmetry: Undirected matrix is always symmetric. Directed is NOT.
- 5. Matrix Golden Rule: `matrix[source][destination] = value`. Row = FROM, Column = TO.
- 6. What Do I Store? List: dest vs (dest, weight). Matrix: 1 vs weight_value.

Universal Graph Builder Template

```
def build_graph(n, edges, directed=False, weighted=False):
    graph = [[] for _ in range(n)]
    for edge in edges:
        u, v = edge[0], edge[1]
        w = edge[2] if weighted else 1
        graph[u].append((v, w) if weighted else v)
        if not directed:
            graph[v].append((u, w) if weighted else u)
    return graph
```

3. PHASE 1 - BFS, DFS & Connectivity

BFS vs DFS - The Two Fundamental Traversals

Aspect	BFS (Breadth-First)	DFS (Depth-First)
Data Structure	Queue (deque, FIFO)	Stack / Recursion (LIFO)
Order	Level by level	Go deep, then backtrack
Shortest Path?	YES (unweighted)	NO
Use Cases	Shortest path, level order	Cycle detection, topo sort
Space	$O(V)$ worst case	$O(V)$ worst case
Time	$O(V + E)$	$O(V + E)$

BFS Template

```
from collections import deque

def bfs(graph, start):
    visited = set([start])
    queue = deque([start])
    while queue:
        node = queue.popleft()
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
```

DFS Template

```
def dfs(graph, node, visited):
    visited.add(node)
    for neighbor in graph[node]:
        if neighbor not in visited:
            dfs(graph, neighbor, visited)
```

Disconnected Graphs - The Outer Loop

If the graph might be disconnected, wrap BFS/DFS in a loop over all nodes:

```
components = 0
visited = set()
for node in range(n):
    if node not in visited:
        dfs(graph, node, visited) # or bfs
        components += 1
```

Phase 1 Problems

1. Count Connected Components

Pattern: DFS outer loop. Count how many times you start a new DFS = number of components.

Trick: Components = number of DFS calls from the outer loop.

2. Number of Provinces (LC 547)

Same as connected components but input is adjacency MATRIX. Scan row for neighbors.

```
# Key difference: neighbors from matrix row
for j in range(n):
    if isConnected[node][j] == 1 and j not in visited:
        dfs(j)
```

3. Number of Islands (LC 200)

2D grid DFS. Each '1' cell is a node. 4-directional neighbors. Flood fill to mark visited.

Count how many times you start a new DFS on an unvisited '1'.

```
directions = [(0,1), (0,-1), (1,0), (-1,0)] # R, L, D, U
for dr, dc in directions:
    nr, nc = r + dr, c + dc
    if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == '1':
        dfs(nr, nc)
```

Grid DFS trick: directions array [(0,1),(0,-1),(1,0),(-1,0)] replaces 4 if-statements.

4. Flood Fill (LC 733)

DFS from starting pixel. Replace old color with new color. Spread to same-color neighbors.

5. Rotting Oranges (LC 994) - Multi-Source BFS

All rotten oranges go into queue FIRST. Each BFS level = 1 minute. Track minutes.

MULTI-SOURCE BFS: "Spread, Infection, Fire, Virus, Distance from nearest" = queue ALL sources first, BFS levels = time.

```
queue = deque()
for r in range(rows):
    for c in range(cols):
        if grid[r][c] == 2: # all rotten oranges
            queue.append((r, c))
minutes = 0
while queue:
    for _ in range(len(queue)): # one BFS level
        r, c = queue.popleft()
        for dr, dc in directions:
            # spread to fresh oranges
    minutes += 1
```

6. Print All Paths (Source to Target)

DFS + Backtracking. Key: set visited[node] = False AFTER recursing to allow other paths.

```
def all_paths(graph, curr, target, visited, path, result):
    visited[curr] = True
    path.append(curr)
    if curr == target:
        result.append(path[:])
    else:
        for neighbor in graph[curr]:
            if not visited[neighbor]:
                all_paths(graph, neighbor, target, visited, path, result)
    path.pop()
    visited[curr] = False # BACKTRACK - the key line
```


4. PHASE 1.5 - Cycle Detection

FAANG Decision Formula: Undirected -> parent tracking. Directed -> recursion stack. Disconnected -> loop all unvisited.

Undirected Graph - 3 Methods

Method 1: DFS + Parent Tracking (Most Common)

If a neighbor is visited AND it's not the parent who sent us here = CYCLE.

```
def has_cycle_undirected(graph, node, visited, parent):
    visited.add(node)
    for neighbor in graph[node]:
        if neighbor not in visited:
            if has_cycle_undirected(graph, neighbor, visited, node):
                return True
        elif neighbor != parent: # visited + not parent = CYCLE
            return True
    return False
```

Why check parent? In undirected, edge A-B means A sees B as visited (B saw A too). That's not a cycle, that's just the edge we came from.

Method 2: BFS + Parent Tracking

Same logic, queue stores (node, parent). If neighbor is visited and not parent = cycle.

Method 3: Union-Find (Best for Dynamic Edges)

For each edge (u, v): if find(u) == find(v) before union = cycle. Best for online queries.

Directed Graph - 2 Methods

Method 1: DFS + Recursion Stack (3-Color)

Two arrays: visited[] and rec_stack[]. A back edge to a node in rec_stack = CYCLE.

Think of 3 colors: WHITE (unvisited), GRAY (in current path), BLACK (fully processed).

```
def has_cycle_directed(graph, node, visited, rec_stack):
    visited[node] = True
    rec_stack[node] = True # GRAY: in current DFS path
    for neighbor in graph[node]:
        if not visited[neighbor]:
            if has_cycle_directed(graph, neighbor, visited, rec_stack):
                return True
        elif rec_stack[neighbor]: # visited + in current path = CYCLE
            return True
    rec_stack[node] = False # BLACK: done, remove from path
    return False
```

Key difference from undirected: We need `rec_stack` (not `parent`). `visited + in_current_path` = back edge = cycle.

Method 2: Kahn's Algorithm (BFS Topological Sort)

If Kahn's processes fewer than V nodes, there's a cycle (stuck nodes have indegree > 0).

Common Mistakes

- DON'T use recursion stack on undirected graphs (unnecessary, parent tracking is enough)
- DON'T use parent tracking on directed graphs (misses cycles)
- DON'T forget the outer loop for disconnected graphs

5. PHASE 2 - Topological Sort & Course Scheduling

What is Topological Sort?

A linear ordering of vertices in a DAG (Directed Acyclic Graph) such that for every edge $u \rightarrow v$, u comes before v in the ordering.

Real-life: course prerequisites, build dependencies, task scheduling.

One-liner: Topological sort = valid order to do tasks when some must come before others. Only works on DAGs (no cycles).

Kahn's Algorithm (BFS Topological Sort) - Preferred in Interviews

The idea: repeatedly remove nodes with no incoming edges (indegree = 0).

```
from collections import deque

def topological_sort(n, edges):
    graph = [[] for _ in range(n)]
    indegree = [0] * n
    for u, v in edges:
        graph[u].append(v)
        indegree[v] += 1

    queue = deque(c for c in range(n) if indegree[c] == 0)
    order = []

    while queue:
        node = queue.popleft()
        order.append(node)
        for neighbor in graph[node]:
            indegree[neighbor] -= 1
            if indegree[neighbor] == 0:
                queue.append(neighbor)

    return order if len(order) == n else [] # empty = cycle
```

Sticky-note analogy: Tasks have sticky notes saying 'wait for X'. Start with tasks that have NO sticky notes (indegree 0). Complete them, peel their notes off others. Repeat.

Phase 2 Problems

1. Course Schedule I (LC 207) - Can all courses be completed?

Convert to: Does the directed graph have a cycle? Use Kahn's. If processed $< n$ = cycle = False.

2. Course Schedule II (LC 210) - Return a valid order

Exact same Kahn's code, but collect order. Return order if len == n, else empty list.

CS-I vs CS-II: Same algorithm. CS-I returns True/False. CS-II returns the order list.

3. Alien Dictionary (LC 269)

Two phases:

Phase 1: Compare adjacent words character by character to find ordering rules (edges).

Phase 2: Topological sort on the character graph.

Edge case: if a longer word comes before its prefix (e.g., 'abc' before 'ab'), return empty.

4. Complete All Courses with Dependencies

Input: arrays a[] and b[] where a[i] must be taken before b[i].

Same as Course Schedule I. Draw a[i] -> b[i], run Kahn's, check completed == n.

6. PHASE 2.5 - Clone Graph Pattern

Clone Graph (LC 133)

Deep copy a graph where nodes can have cycles. HashMap maps original -> clone.

One-liner: "HashMap is your MEMORY - prevents duplicates AND handles cycles."

```
def clone_graph(node):
    if not node: return None
    hashmap = {}

    def dfs(node):
        if node in hashmap:
            return hashmap[node] # already cloned
        copy = Node(node.val)
        hashmap[node] = copy    # store BEFORE recursing!
        for neighbor in node.neighbors:
            copy.neighbors.append(dfs(neighbor))
        return copy

    return dfs(node)
```

Critical Detail

Store clone in hashmap BEFORE recursing into neighbors. Otherwise, cycles cause infinite recursion.

Same Pattern Appears In

- Clone Graph (LC 133)
- Copy List with Random Pointer (LC 138)
- Deep Copy Binary Tree (LC 1485)
- Clone N-ary Tree (LC 1490)

Rule: "Deep copy + cycles or back-references = HashMap pattern"

7. PHASE 3 - Shortest Path Algorithms

The Decision Tree (Memorize This!)

```

Need shortest path?
|
+-- All pairs? --> YES --> Floyd-Warshall (V <= 400)
|
+-- NO (single source)
    |
    +-- Negative weights? --> YES --> Bellman-Ford
    |
    +-- NO
        |
        +-- Weighted? --> YES --> Dijkstra
        |
        +-- NO --> BFS
  
```

Quick Rule: "No weights? BFS. Positive weights? Dijkstra. Negative? Bellman-Ford. All pairs? Floyd-Warshall."

Algorithm	Graph Type	Time	Space
BFS	Unweighted	$O(V + E)$	$O(V)$
Dijkstra	Weighted (positive)	$O((V+E) \log V)$	$O(V)$
Bellman-Ford	Weighted (negative ok)	$O(V * E)$	$O(V)$
Floyd-Warshall	All pairs	$O(V^3)$	$O(V^2)$

Algorithm 1: BFS Shortest Path (Unweighted)

First arrival at a node via BFS = shortest distance. Each edge has weight 1.

```

def bfs_shortest(graph, src, n):
    dist = [-1] * n
    dist[src] = 0
    queue = deque([src])
    while queue:
        node = queue.popleft()
        for neighbor in graph[node]:
            if dist[neighbor] == -1:
                dist[neighbor] = dist[node] + 1
                queue.append(neighbor)
    return dist
  
```

Algorithm 2: Dijkstra (Positive Weights Only)

Greedy: always process the cheapest unvisited node. Uses a min-heap.

Dijkstra in one line: "BFS with a priority queue, always picking the cheapest node first."

Heart of Dijkstra = RELAXATION:

if $\text{dist}[\text{node}] + \text{weight} < \text{dist}[\text{neighbor}]$: update $\text{dist}[\text{neighbor}]$

```
import heapq

def dijkstra(graph, src, n):
    dist = [float('inf')] * n
    dist[src] = 0
    heap = [(0, src)] # (cost, node)

    while heap:
        cost, node = heapq.heappop(heap)
        if cost > dist[node]: # stale entry, skip
            continue
        for neighbor, weight in graph[node]:
            new_dist = dist[node] + weight
            if new_dist < dist[neighbor]:
                dist[neighbor] = new_dist
                heapq.heappush(heap, (new_dist, neighbor))
    return dist
```

Why Dijkstra Fails with Negative Weights

Dijkstra is greedy - once a node is popped, it's considered final. But a negative edge discovered later could provide a shorter path to an already-finalized node. Dijkstra would miss it.

Algorithm 3: Bellman-Ford (Handles Negative Weights)

Relax ALL edges, V-1 times. Guaranteed to find shortest paths.

One-liner: "Relax ALL edges V-1 times. If the V-th pass still relaxes, there is a negative cycle."

Why V-1 iterations? The longest shortest path has at most V-1 edges. Each iteration propagates the shortest path by one more edge.

```
def bellman_ford(n, edges, src):
    dist = [float('inf')] * n
    dist[src] = 0

    for _ in range(n - 1):          # V-1 iterations
        for u, v, w in edges:
            if dist[u] != float('inf') and dist[u] + w < dist[v]:
                dist[v] = dist[u] + w

    # Negative cycle detection (V-th pass)
    for u, v, w in edges:
        if dist[u] != float('inf') and dist[u] + w < dist[v]:
            return None # negative cycle!
    return dist
```

LC 787: Cheapest Flights Within K Stops (Modified Bellman-Ford)

Run K+1 rounds instead of V-1. Use dist.copy() each round to prevent chaining.

The COPY is the key! Without dist.copy(), one round chains multiple flights, exceeding K stops.

```
def findCheapestPrice(n, flights, src, dst, k):
    dist = [float('inf')] * n
    dist[src] = 0

    for _ in range(k + 1):        # K stops = K+1 flights
        prev = dist[:]           # COPY prevents chaining
        for u, v, w in flights:
            if prev[u] + w < dist[v]:
                dist[v] = prev[u] + w
    return dist[dst] if dist[dst] != float('inf') else -1
```

Algorithm 4: Floyd-Warshall (All Pairs)

For every pair (i,j), try every node k as a middle-man. Three nested loops.

Core: $dist[i][j] = \min(dist[i][j], dist[i][k] + dist[k][j])$. K is the OUTER loop!


```
def floyd_warshall(n, dist):
    # dist[i][j] = edge weight or inf
    for k in range(n):          # k = intermediate node (OUTER!)
        for i in range(n):
            for j in range(n):
                if dist[i][k] + dist[k][j] < dist[i][j]:
                    dist[i][j] = dist[i][k] + dist[k][j]
    # Negative cycle: dist[i][i] < 0 for some i
    return dist
```

Why K Must Be the Outer Loop (Interviewers Ask This!)

k represents 'allowed intermediates so far.' With k as outer loop, when we consider node k as a shortcut, we've already computed shortest paths using nodes 0..k-1. If k were inner, we'd try shortcuts before knowing if they're optimal.

Snakes and Ladders (LC 909) - BFS Application

Board cell = node. Dice roll = edge (up to 6 per node). Snake/ladder = forced teleport.

Minimum dice rolls = shortest path in unweighted graph = BFS.

Each BFS level = one dice roll. Mark TELEPORT DESTINATION visited (not the snake/ladder cell). You never stay on a teleport cell.

```
def snakes_and_ladders(n, moves):
    queue = deque([0]) # 0-indexed
    visited = [False] * n
    visited[0] = True
    rolls = 0
    while queue:
        for _ in range(len(queue)): # one BFS level = one roll
            curr = queue.popleft()
            if curr == n - 1: return rolls
            for dice in range(1, 7):
                nxt = curr + dice
                if nxt >= n: continue
                if moves[nxt] != -1: nxt = moves[nxt] # teleport
                if not visited[nxt]:
                    visited[nxt] = True
                    queue.append(nxt)
            rolls += 1
    return -1
```

8. PHASE 4 - Minimum Spanning Tree (MST)

What is an MST?

A subset of edges that connects ALL vertices with minimum total weight, using exactly $V-1$ edges, with no cycles. Think: cheapest way to wire all houses together.

3 Must-Know Properties: Spanning (all vertices), Tree ($V-1$ edges, no cycles), Minimum (smallest total weight).

Prim's vs Kruskal's

Aspect	Prim's	Kruskal's
Approach	Grow tree from one node	Pick cheapest edges globally
Data Structure	Min-Heap + visited[]	Sort edges + Union-Find
Best For	Dense graphs ($E \sim V^2$)	Sparse graphs ($E \sim V$)
Time	$O((V+E) \log V)$	$O(E \log E)$
Edge list given?	Build adj list first	Use directly

Prim's = Dijkstra but push EDGE WEIGHT, not total distance. Kruskal's = Sort edges, pick cheapest, skip if cycle (Union-Find).

Prim's Algorithm

```
import heapq

def prims(graph, n):
    visited = [False] * n
    heap = [(0, 0)] # (weight, start_node)
    total = 0
    edges_used = 0

    while heap and edges_used < n:
        weight, node = heapq.heappop(heap)
        if visited[node]: continue
        visited[node] = True
        total += weight
        edges_used += 1
        for neighbor, w in graph[node]:
            if not visited[neighbor]:
                heapq.heappush(heap, (w, neighbor))
    return total
```

Kruskal's Algorithm + Union-Find

```
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x]) # path compression
        return self.parent[x]

    def union(self, x, y):
        px, py = self.find(x), self.find(y)
        if px == py: return False # already connected = cycle
        if self.rank[px] < self.rank[py]: px, py = py, px
        self.parent[py] = px
        if self.rank[px] == self.rank[py]: self.rank[px] += 1
        return True

def kruskals(n, edges):
    edges.sort(key=lambda x: x[2]) # sort by weight
    uf = UnionFind(n)
    total, count = 0, 0
    for u, v, w in edges:
        if uf.union(u, v): # no cycle
            total += w
            count += 1
            if count == n - 1: break
    return total
```

LC 1584: Min Cost to Connect All Points

Complete graph (every pair connected) with Manhattan distance $|x_1 - x_2| + |y_1 - y_2|$. Use Prim's (dense graph).

9. PHASE 5 - Advanced DFS (Bridges & Articulation Points)

Tarjan's Algorithm - Two Key Arrays

$disc[node]$ = discovery timestamp (WHEN did DFS find this node?)

$low[node]$ = lowest disc reachable from this node's subtree via back edges.

Memory trick: disc = "discovery TIME". low = "how LOW (early) can I reach?"

Bridges (Critical Connections) - LC 1192

An edge (parent, child) is a bridge if removing it disconnects the graph.

Condition: $low[child] > disc[parent]$ (strictly greater).

Meaning: child's subtree has NO back edge to parent or above. Removing the edge isolates the subtree.

Bridge analogy: "Can my child find its way back to me without our direct edge? If NO, our edge is a bridge."

```
def find_bridges(graph, n):
    disc = [-1] * n
    low = [-1] * n
    bridges = []
    timer = [0]

    def dfs(node, parent):
        disc[node] = low[node] = timer[0]
        timer[0] += 1
        for neighbor in graph[node]:
            if disc[neighbor] == -1:          # unvisited (tree edge)
                dfs(neighbor, node)
                low[node] = min(low[node], low[neighbor])
                if low[neighbor] > disc[node]: # BRIDGE
                    bridges.append((node, neighbor))
            elif neighbor != parent:          # back edge
                low[node] = min(low[node], disc[neighbor])

    for i in range(n):
        if disc[i] == -1:
            dfs(i, -1)
    return bridges
```

Articulation Points

A vertex whose removal disconnects the graph.

Condition: $low[child] \geq disc[node]$ (greater or EQUAL) + special case: root with 2+ DFS children.

Bridge: > (strictly greater). Articulation Point: $\geq$ (greater or equal). That's the ONLY difference in the condition.

Updating low[] - The Rules

- Tree edge (unvisited child): $\text{low}[\text{node}] = \min(\text{low}[\text{node}], \text{low}[\text{child}])$
- Back edge (visited ancestor, not parent): $\text{low}[\text{node}] = \min(\text{low}[\text{node}], \text{disc}[\text{ancestor}])$

IMPORTANT: Use disc[ancestor], NOT low[ancestor] for back edges!

10. FAANG PROBLEM SET - 30+ Must-Solve by Phase

Phase 1: BFS / DFS / Connectivity

LC #	Problem	Pattern
200	Number of Islands	DFS flood fill on grid
547	Number of Provinces	DFS on adjacency matrix
733	Flood Fill	DFS color replacement
994	Rotting Oranges	Multi-source BFS
695	Max Area of Island	DFS + area tracking
1091	Shortest Path in Binary Matrix	BFS on grid
127	Word Ladder	BFS word transformation
542	01 Matrix	Multi-source BFS
286	Walls and Gates	Multi-source BFS
797	All Paths From Source to Target	DFS + backtracking

Phase 1.5: Cycle Detection

LC #	Problem	Pattern
--	Cycle in Undirected Graph	DFS + parent tracking
--	Cycle in Directed Graph	DFS + recursion stack
684	Redundant Connection	Union-Find cycle detection

Phase 2: Topological Sort

LC #	Problem	Pattern
207	Course Schedule I	Kahn's BFS (cycle detection)
210	Course Schedule II	Kahn's BFS (return order)
269	Alien Dictionary	Build graph + topo sort
--	Complete All Courses (IK)	Same as LC 207

Phase 2.5: Clone Graph

LC #	Problem	Pattern
133	Clone Graph	HashMap + DFS
138	Copy List with Random Pointer	Same HashMap pattern

Phase 3: Shortest Path

LC #	Problem	Pattern
743	Network Delay Time	Dijkstra (single source)
787	Cheapest Flights Within K Stops	Modified Bellman-Ford / Dijkstra
1631	Path With Minimum Effort	Dijkstra on grid

778	Swim in Rising Water	Dijkstra / Binary Search + BFS
1334	Find City With Smallest Neighbors	Floyd-Warshall
909	Snakes and Ladders	BFS (unweighted shortest path)
1976	Number of Ways to Arrive at Destination	Dijkstra + counting
847	Shortest Path Visiting All Nodes	BFS + bitmask

Phase 4: Minimum Spanning Tree

LC #	Problem	Pattern
1584	Min Cost to Connect All Points	Prim's (dense/complete graph)
1135	Connecting Cities With Min Cost	Kruskal's / Prim's
1168	Optimize Water Distribution	Virtual node + MST
684	Redundant Connection	Union-Find

Phase 5: Advanced DFS

LC #	Problem	Pattern
1192	Critical Connections in Network	Tarjan's bridges
--	Articulation Points	Tarjan's ($\geq$ condition)

11. MASTER CHEAT SHEET - Tricks, Templates & Decision Trees

15 Memory Tricks to Remember Forever

- 1. "Direction decides HOW MANY appends. Weight decides WHAT you append."
- 2. "Mirror Rule: Directed = 1 append. Undirected = 2 appends."
- 3. "HashMap is your MEMORY - prevents duplicates AND handles cycles." (Clone Graph)
- 4. "Prim = Dijkstra but push EDGE WEIGHT, not total distance."
- 5. "Kruskal = Sort edges, pick cheapest, skip if Union-Find says cycle."
- 6. "No weights? BFS. Positive? Dijkstra. Negative? Bellman-Ford."
- 7. "Relax ALL edges V-1 times. V-th pass still relaxes = negative cycle."
- 8. "Floyd-Warshall: 3 nested loops k-i-j. K must be OUTER."
- 9. "disc = discovery TIME. low = how LOW can I reach?"
- 10. "Can my child find way back without our edge? No = bridge."
- 11. "Bridge: >. Articulation Point: >=. Only difference."
- 12. "Undirected cycle: parent tracking. Directed cycle: recursion stack."
- 13. "Spread/Infection/Fire = Multi-source BFS (queue ALL sources first)."
- 14. "Deep copy + cycles = HashMap pattern."
- 15. "Bellman-Ford + K+1 rounds + dist.copy() = flights with K stops."

Pattern Recognition Quick Reference

When you see...	Think this...
"Minimum steps/moves"	BFS (unweighted shortest path)
"Cheapest/shortest with weights"	Dijkstra
"With negative weights"	Bellman-Ford
"All pairs shortest"	Floyd-Warshall ($V \leq 400$)
"Must do X before Y"	Topological Sort (Kahn's)
"Is valid ordering possible?"	Cycle detection
"Count connected groups"	DFS/BFS outer loop
"Spread/infection/fire"	Multi-source BFS
"Grid traversal"	DFS/BFS with directions array
"Deep copy graph/list"	HashMap (original -> clone)
"Cheapest to connect all"	MST (Prim's or Kruskal's)
"Removing edge disconnects?"	Bridges (Tarjan's)
"Removing node disconnects?"	Articulation Points (Tarjan's)
"With limited stops/hops"	Modified Bellman-Ford (K+1 rounds)
"Board game, min dice rolls"	BFS (each level = one roll)

Complexity Quick Reference

Algorithm	Time	Space	Notes
BFS / DFS	$O(V+E)$	$O(V)$	Foundation

Topological Sort	$O(V+E)$	$O(V+E)$	DAGs only
Dijkstra (heap)	$O((V+E)\log V)$	$O(V)$	No neg weights
Bellman-Ford	$O(V \cdot E)$	$O(V)$	Handles neg weights
Floyd-Warshall	$O(V^3)$	$O(V^2)$	All pairs, $V \leq 400$
Prim's (heap)	$O((V+E)\log V)$	$O(V)$	Dense graphs
Kruskal's	$O(E \log E)$	$O(V)$	Sparse graphs
Tarjan's	$O(V+E)$	$O(V)$	Bridges / APs
Union-Find	$O(\alpha(N))$	$O(V)$	Nearly $O(1)$

The Ultimate Interview Checklist

- Can I identify the graph type? (directed/undirected, weighted/unweighted)
- Do I know the representation? (adjacency list default, matrix if asked)
- Can I handle disconnected components? (outer loop over all nodes)
- Can I detect cycles? (parent for undirected, rec_stack for directed)
- Can I find shortest path? (BFS/Dijkstra/Bellman-Ford decision tree)
- Can I find MST? (Prim's for dense, Kruskal's for sparse)
- Can I find bridges/APs? (Tarjan's disc[] and low[])
- Can I do topological sort? (Kahn's with indegree queue)
- Can I clone a graph? (HashMap prevents cycles and duplicates)
- Can I explain TIME and SPACE complexity for each algorithm?